.NET Developer
to Enterprise Architect

The Complete Handbook — Career Roadmap · Study Guide · Interview Prep · Architecture Reference

35 sections covering: CS Fundamentals · C# from scratch · OOP · SOLID · Patterns · Async · EF Core · Clean Architecture
CQRS · DDD · Testing · Git · CI/CD · Docker · Kubernetes · Azure · Security · JWT · Redis · Service Bus
System Design · Multi-tenancy · Microservices · Observability · DevOps · SA · EA · Intent Architect

Bongani Nyoni · Senior .NET Developer · CPUT BEng Tech Honours · Ikamva Tech Lab

MASTER FLOW MAP

How everything fits: code to production

Every stage below is covered in this deck. Tap any stage to understand it in detail.

C# Code
Clean Arch + CQRS

Git + PR
Branch → Review → Merge

CI Pipeline
Build · Test · Scan

Docker Image
Multi-stage build

Container Registry
ACR

CD Pipeline
Dev → Staging → Prod

AKS / App Service
Pods · Ingress · HPA

API Management
Gateway · Auth · Throttle

Security Layer
JWT · AD · Key Vault

Azure Services
Bus · Redis · Blob

Azure SQL / Cosmos
Data layer

Redis Cache
Cache-aside · TTL

Service Bus
Queues · Topics · DLQ

App Insights
Traces · Logs · Alerts

Production
Monitor · Support · SRE

↓ Architecture decision sits above everything — it determines what this diagram looks like ↓

CS Fundamentals

Algorithms · Data Structures · Big O · Where they matter in enterprise .NET

CS FUNDAMENTALS

Data structures & Big O notation

Big O describes how an algorithm's time or memory grows as input grows. Knowing this prevents production performance disasters.

Big O — the 6 you must know

O(1)	**Constant** Dictionary lookup, array index	*Lightning fast — no matter how big the data*
O(log n)	**Logarithmic** Binary search, balanced BST	*Doubles input = one extra step*
O(n)	**Linear** LINQ .Where(), List.Contains	*One step per element — scales with data*
O(n log n)	**Linearithmic** Sorting (LINQ .OrderBy())	*Very acceptable for sorting*
O(n²)	**Quadratic** Nested foreach loops	*Avoid — 1000 items = 1M operations*
O(2ⁿ)	**Exponential** Naive recursive Fibonacci	*Never in production code*

Data structures — choose wisely

Wrong collection = hidden O(n²) in production. Choose based on your dominant operation.

Array / List<T> Get O(1) · Add O(1)* · Insert O(n)	*Ordered sequence, index access*
Dictionary<K,V> Get/Set/Contains O(1)	*Lookup by key — use for caches, lookups*
HashSet<T> Contains O(1) · Add O(1)	*Unique values, fast membership test*
SortedDictionary Get O(log n) · Iterate sorted	*Need sorted keys — e.g. priority queues*
LinkedList<T> Insert/Delete O(1) with ref	*Frequent middle insertions/deletions*
Queue<T> Enqueue/Dequeue O(1)	*FIFO processing — background job queues*
Stack<T> Push/Pop O(1)	*LIFO — undo systems, expression parsing*

Where Big O matters in enterprise .NET

C# Fundamentals

Types · Memory · OOP · SOLID · Async · LINQ

Value types, reference types & memory

Understanding stack vs heap prevents memory leaks, boxing overhead, and subtle bugs in high-throughput .NET services.

Stack vs Heap

The stack is fast, auto-cleaned. The heap requires GC. Misusing them causes either GC pressure or stack overflow.

```
// Stack — value types, auto-managed
int age = 30;           // 4 bytes on stack
double price = 9.99;    // 8 bytes on stack
bool active = true;     // 1 byte on stack
// When method exits, stack frame is popped — instant cleanup

// Heap — reference types, GC-managed
var customer = new Customer(); // reference on stack
                               // Customer object on heap
string name = "Bongani";       // string is on heap (immutable)
```

- struct = value type on stack, class = reference type on heap
- record = immutable reference type with value equality (C#9+)
- record struct = immutable value type (C#10+)
- IDisposable: deterministic cleanup of unmanaged resources (DB connections, streams)
- using statement = guaranteed Dispose() even if exception thrown
- GC generations: Gen0 (short-lived), Gen1, Gen2 (long-lived) — avoid Gen2 pressure

Modern C# type features

```
// Nullable reference types (C#8+) — opt-in null safety
string  name  = "Bongani"; // non-nullable — compiler warns if null assigned
string? email = null;      // nullable — explicit intent

// Records — immutable, value equality, with-expressions
public record Customer(int Id, string Name, string Email);
var c1 = new Customer(1, "Bongani", "b@test.com");
var c2 = c1 with { Email = "new@test.com" }; // copy with mutation

// Pattern matching (C#9+)
var label = order.Status switch {
    OrderStatus.Pending   => "Awaiting payment",
    OrderStatus.Shipped   => "On the way",
    OrderStatus.Delivered when order.Total > 1000 => "VIP delivered",
    _                     => "Unknown"
};
```

- const = compile-time constant; readonly = set once at construction
- Nullable types: int? — adds null to any value type — use sparingly
- Span<T> and Memory<T>: slice without allocation — essential for performance code
- Primary constructors (C#12): class OrderService(IOrderRepository repo)

OOP — FOUR PILLARS

OOP: the four pillars with explanations

Encapsulation hides state. Abstraction hides complexity. Inheritance shares behaviour. Polymorphism lets one interface serve many types.

Encapsulation

Protect internal state. Only expose what callers need. Prevents your object from being put in an invalid state by outside code.

- private fields + public properties — not public fields
- private set: read anywhere, write only inside the class
- Expose behaviour not data: account.Deposit(100) not account.Balance += 100
- Access modifiers: private < protected < internal < public
- Benefits: invariants are always maintained; refactor internals without breaking API
- Violation: god objects, public fields, anemic domain models

Abstraction

Show only the interface; hide the implementation. You use List<T> without knowing how it resizes internally.

- interface = pure contract: what, not how
- abstract class = partial implementation + forced contract
- Program to the abstraction: IOrderRepository not SqlOrderRepository
- Benefit: swap implementations without changing callers
- Interfaces enable unit testing — inject a fake instead of a real DB
- Every dependency should be expressed as an interface

Inheritance

Share code between related types. A child automatically has all members of its parent. Use sparingly — prefer composition.

- class Dog : Animal — Dog IS-A Animal; gets all Animal members
- virtual + override: child provides its own version of parent method
- abstract method: parent declares shape, child must implement
- sealed: prevents further subclassing (string, Int32 are sealed)
- base.Method(): call parent version from child
- Prefer composition: class Car { Engine _e; } over class Car : Engine

Polymorphism

Many forms. One variable of type Animal can point to a Dog or Cat. The correct method is chosen at runtime.

- Runtime (dynamic): List<Shape> — Shape.Area() calls the right override
- Interface: IOrderRepository repo = new SqlRepo() or new FakeRepo()
- is/as for safe downcasting: if (a is Dog d) { d.Fetch(); }
- switch expression with patterns: o switch { Dog d => ..., Cat c => ... }
- Polymorphism + interfaces = the foundation of testable, extensible code
- This is WHY you inject IService, not the concrete class

SOLID: S — Single Responsibility · O — Open/Closed

SOLID is 5 principles that make code easier to change and test. Violating them creates code that breaks in unexpected places when you change something

S — Single Responsibility Principle

One class = one reason to change. If adding a new email provider forces you to change OrderService, it has two responsibilities.

```
// VIOLATES SRP — 3 reasons to change
class OrderService {
  void PlaceOrder(Order o) { /* business logic */ }
  void SendEmail(Order o)  { /* email logic — reason 2 */ }
  void SaveToDb(Order o)   { /* DB logic — reason 3 */ }
}

// CORRECT — one reason each
class OrderService    { void PlaceOrder(Order o) { } }
class OrderEmailer    { void SendEmail(Order o) { } }
class OrderRepository { void SaveToDb(Order o) { } }
```

- If you can't name the class in one noun, it probably does too much
- A class with >200 lines is a smell — consider splitting
- SRP at layer level: Domain / Application / Infrastructure are separate
- In Clean Architecture SRP is enforced structurally by the folder layout
- Tests: SRP classes are easy to test — few dependencies, clear purpose
- Interview: 'Give me an example of SRP violation you fixed'

O — Open/Closed Principle

Open for extension, closed for modification. Add new behaviour by adding new code, not by editing tested existing code.

```
// VIOLATES OCP — adding VIP discount = edit this file
decimal CalcDiscount(Order o, string type) {
  if(type=="loyalty") return o.Total * 0.1m;
  if(type=="bulk")    return o.Total * 0.15m;
  // editing tested code = risk of regression
}

// CORRECT — add new discount = add new class
interface IDiscountStrategy {
  decimal Calculate(Order o);
}
class LoyaltyDiscount : IDiscountStrategy { ... }
class BulkDiscount    : IDiscountStrategy { ... }
// New: VipDiscount: add VipDiscount class, touch nothing
```

- Achieved through: interfaces, abstract classes, strategy pattern
- MediatR: new command = new handler file — never edit existing handlers
- New type = new class, not a new if/else branch
- Existing code is tested — don't break it to add features
- OCP makes large codebases safe to extend — essential at enterprise scale
- Interview: 'How does the Strategy pattern enforce OCP?'

SOLID — LSP · ISP · DIP

SOLID: L — Liskov · I — Interface Segregation · D — Dependency Inversion

These three principles ensure subclasses behave correctly, interfaces stay focused, and business logic never depends on infrastructure.

L — Liskov Substitution

If Dog inherits Animal, you must be able to use Dog wherever Animal is expected without surprising or breaking anything.

- Subclass must not STRENGTHEN preconditions — accept at least as much as parent
- Subclass must not WEAKEN postconditions — return at least as much as parent
- Classic violation: Square extends Rectangle but breaks SetWidth behaviour
- Test: if you have 'if(x is SubType)' workarounds, LSP is violated
- Practical: design for 'is-a' relationships carefully — Square IS-NOT-A Rectangle behaviourally
- Interview: 'Describe an LSP violation you have seen'

I — Interface Segregation

Don't force a class to implement methods it doesn't need. Split large interfaces into focused ones.

- Large interface with 10 methods = most classes implement 3 and throw on the rest
- Split by client need: IOrderReader / IOrderWriter instead of IOrderRepository
- Common in .NET: IEnumerable < ICollection < IList — each layer adds more
- Smaller interfaces = easier to mock in unit tests
- Real example: you inject IOrderReader in a read-only handler — it can't accidentally write
- Interview: 'Why is ISP important for testability?'

D — Dependency Inversion

High-level (business logic) must not depend on low-level (SQL, SMTP), both depend on abstractions (interfaces).

- OrderService depends on IOrderRepository, not SqlOrderRepository
- The interface is defined in the Domain/Application layer — not Infrastructure
- Infrastructure layer depends inward — never the other way
- This is WHY Clean Architecture and DI containers exist
- Testing benefit: inject FakeRepository — no DB needed in unit tests
- Interview: 'What is the difference between DI and DIP?'

SOLID applied: the before and after

```
// BEFORE (all 5 violated): one class creates SQL directly, sends email, has Fax() that throws, if/else for types, breaks parent contract
// AFTER (all 5 applied): OrderService(IOrderRepository, IEmailService, IDiscountStrategy) — testable, extendable, each class one reason to change
```

ENGINEERING PRINCIPLES

DRY · KISS · YAGNI · Fail Fast · Idempotency & more

These principles are the 'common sense' of software engineering. Violating them creates fragile, over-engineered, hard-to-maintain systems.

DRY · KISS · YAGNI

The three principles that prevent over-engineering and duplication — the most common mistakes junior developers make.

- DRY: Don't Repeat Yourself — duplicated logic = two places to fix bugs
- KISS: Keep It Simple — the simplest solution that solves the problem wins
- YAGNI: You Aren't Gonna Need It — don't build features before they're needed
- DRY violation: copy-pasting business logic across handlers
- KISS violation: microservices for a 3-table CRUD app
- YAGNI violation: building a plugin system before you have 2 plugins
- Balance: DRY can be taken too far — wrong abstraction is worse

Fail Fast · Defensive Programming

Fail fast means detect errors as early as possible. Defensive programming means never trust input.

- Fail fast: validate at the entry point — don't let bad data travel deep
- Guard clauses: if (id <= 0) throw new ArgumentException() at top of method
- FluentValidation: validate command before handler runs (pipeline behaviour)
- Defensive: never assume strings are non-null, never assume enums are valid
- Null object pattern: return empty list not null — callers don't need null checks
- ArgumentNullException.ThrowIfNull(repo) in constructor — fail at registration time

Idempotency · Immutability

Idempotent operations can be safely retried. Immutable data cannot be corrupted by accident. Critical for distributed systems.

- Idempotency: same request sent N times = same result as sending it once
- Required for: Service Bus consumers, payment APIs, retry policies (Polly)
- Achieve with: idempotency key header, check-before-insert, upsert
- Immutability: record types, readonly fields, init-only properties
- Immutable value objects: Money, Address — equality by value, no mutation
- Thread safety: immutable objects need no locking — safe for concurrency
- Interview: 'How do you make a Service Bus consumer idempotent?'

Twelve-Factor App · Separation of Concerns

Twelve-Factor is the definitive guide to cloud-native app design. Separation of Concerns is why we have layers at all.

- Factor 3: Config in environment — never hardcode; use Key Vault / App Settings
- Factor 6: Stateless processes — no session state in memory; use Redis
- Factor 7: Port binding — app exposes a port; doesn't depend on runtime container
- Factor 9: Disposability — fast startup, graceful shutdown (SIGTERM handling)
- SoC: each layer has one concern — API handles HTTP, Domain handles business rules
- Law of Demeter: only talk to direct dependencies — 'don't talk to strangers'
- Interview: 'Which Twelve-Factor principles apply to your

ASYNC / AWAIT

Async/await: the mental model and the rules

async/await is about freeing threads during I/O waits, not about running things in parallel. Understanding this prevents deadlocks and thread pool starvation.

How it really works

Like a waiter: takes your order, doesn't stand at the kitchen waiting — serves other tables. await = walking away; callback = food's ready.

```
// WRONG — blocks the thread, causes deadlock in
ASP.NET
public Order GetOrder(int id) {
  return _repo.GetByIdAsync(id).Result; // DEADLOCK
RISK
}

// CORRECT — thread freed while DB is queried
public async Task<Order?> GetOrderAsync(int id) {
  return await _repo.GetByIdAsync(id);
  // thread returned to pool at 'await'
  // method resumes here when DB responds
}

// Parallel — run multiple I/O operations
simultaneously
public async Task<DashboardDto>
GetDashboardAsync(int id) {
  var ordersTask   = _repo.GetOrdersAsync(id);
  var customerTask = _repo.GetCustomerAsync(id);
  await Task.WhenAll(ordersTask, customerTask); //
parallel!
  return new DashboardDto(ordersTask.Result,
customerTask.Result);
}
```

The 7 rules

- NEVER .Result or .Wait() — causes deadlocks in ASP.NET Core
- async void only for event handlers — exceptions vanish otherwise
- Always propagate CancellationToken — enables graceful cancellation
- ConfigureAwait(false) in library code — avoids unnecessary context capture
- async Task not async void — makes exceptions observable
- IAsyncEnumerable<T> for streaming large datasets from DB
- ValueTask<T> for hot-path operations that are usually synchronous

Advanced patterns

```
// Limit concurrency — don't hammer the database
var semaphore = new SemaphoreSlim(5); // max 5
concurrent
var tasks = items.Select(async item => {
  await semaphore.WaitAsync(ct);
  try   { return await ProcessAsync(item, ct); }
  finally { semaphore.Release(); }
});
await Task.WhenAll(tasks);

// Stream large results — don't load all into
memory
await foreach (var order in
_repo.StreamOrdersAsync(filter, ct))
  await ProcessOneAsync(order, ct);

// Channel — high-performance producer/consumer
var ch = Channel.CreateBounded<Order>(capacity:100);
// Producer:
await channel.Writer.WriteAsync(order, ct);
// Consumer:
await foreach (var o in
channel.Reader.ReadAllAsync(ct))
  await ProcessAsync(o, ct);
```

Interview questions

- Q: What is the difference between Task and ValueTask?
  A: Task allocates on heap; ValueTask avoids allocation when result is synchronous — use in hot paths
- Q: What causes a deadlock with async/await?
  A: Calling .Result on a Task in a context with a SynchronizationContext (old ASP.NET, WinForms)
- Q: How do you cancel an async operation?
  A: Pass CancellationToken through every async method; call token.ThrowIfCancellationRequested()

LINQ: query anything with the same syntax

LINQ (Language Integrated Query) works on any IEnumerable<T> — in-memory lists, EF Core (SQL), XML, or custom data sources. Deferred execution is the key

Core operators with use cases

```
var result = orders
  .Where(o => o.Status == "Active")          // filter — O(n)
  .OrderByDescending(o => o.Total)           // sort — O(n log n)
  .GroupBy(o => o.CustomerId)                // group — O(n)
  .Select(g => new OrderSummaryDto {         // transform
(projection)
    CustomerId = g.Key,
    OrderCount = g.Count(),
    TotalSpent = g.Sum(o => o.Total),
    LastOrder  = g.Max(o => o.CreatedAt)
  })
  .Take(10)                                  // limit
  .ToList();                                 // ← MATERIALISE
HERE (execute)

// SelectMany — flatten nested collections
var allItems = orders.SelectMany(o => o.Items);
```

- EF Core translates Where/Select/OrderBy/GroupBy to SQL — runs on DB server
- Client eval: calling a C# method inside Where() = LOAD ALL ROWS first — dangerous
- AsNoTracking(): 30-40% faster for read-only EF queries — no change tracking
- FirstOrDefault() is safe; First() throws if empty; Single() throws if more than 1
- Any() stops at first match; Count() > 0 scans all — use Any() for membership test
- Prefer Select() projections — load only columns you need, not full entities

LINQ interview Q&A

What is deferred execution?

Query builds a description; runs only when enumerated. ToList() forces execution.

Difference between Select and SelectMany?

Select transforms each element 1:1. SelectMany flattens one-to-many (Order → Items).

Why is Any() better than Count() > 0?

Any() stops at first match — O(1) best case. Count() always scans all — O(n).

What is N+1 problem with EF LINQ?

Loading orders then foreach loading customer = N+1 DB queries. Fix: .Include(o => o.Customer).

When does EF LINQ break?

When you call C# methods inside Where() that EF can't translate to SQL — moves to client eval.

DESIGN PATTERNS — CREATIONAL

Design patterns: Factory · Abstract Factory · Builder · Singleton

Creational patterns separate object creation from usage. They make systems flexible and testable.

Factory Method

Create objects through an interface without exposing the concrete type. Callers never use 'new ConcreteType()' directly.

- IShapeFactory.Create('circle') returns IShape — caller doesn't know which
- ServiceFactory.CreateForTenant(tenantType) — silo vs pool strategy
- Benefit: change the created type without changing callers
- In .NET: IHttpClientFactory — creates configured HttpClient instances
- Design: single Create method returning an interface
- Interview: 'Why is new() a dependency and how do you remove it?'

Builder

Construct complex objects step by step. The same construction process can produce different representations.

- new OrderBuilder().ForCustomer(1).WithItem(p1,2).WithDiscount('PROMO').Build()
- Fluent API: each method returns this — enables method chaining
- Used in: WebApplicationBuilder, EF Core ModelBuilder, HttpClient configuration
- Benefit: readable construction of complex objects with many optional fields
- Vs constructor: builder is clearer when you have >4 parameters
- Interview: 'When would you choose Builder over a

Singleton

Ensure only one instance exists. In .NET, the DI container manages this — you rarely implement it manually.

- AddSingleton<T>() — one instance for the lifetime of the application
- Use for: configuration, caches, HTTP clients, connection pools, feature flags
- DANGER: Singleton that holds mutable state must be thread-safe
- NEVER inject Scoped service into Singleton — captive dependency bug
- Do NOT implement manually in .NET — let the DI container manage it
- Interview: 'What is the captive dependency problem?'

Abstract Factory

Create families of related objects without specifying their concrete classes. Ensures compatibility between platforms.

- IPaymentGatewayFactory.CreateForRegion('ZA') vs CreateForRegion('EU')
- ITenantServicesFactory: SiloServices vs PoolServices per tenant model
- Benefit: consistent object families — all SA objects use SA rules together
- Different from Factory Method: creates families, not single objects
- Real example: EF Core uses this for different database providers
- Interview: 'Difference between Factory Method and Abstract Factory?'

DESIGN PATTERNS — STRUCTURAL & BEHAVIORAL

Patterns: Decorator · Strategy · Observer · Mediator · Command

These patterns show up constantly in enterprise .NET — Decorator in MediatR pipelines, Observer in domain events, Strategy in discount calculations.

Decorator

Add behaviour to an object without changing its class. Wrap it in another object with the same interface.

- LoggingRepository wraps SqlRepository — adds logging, SqlRepository unchanged
- MediatR pipeline behaviours ARE Decorator — each wraps the next
- Decorator = Open/Closed in practice: add logging by adding a class
- Chain decorators: SqlRepository → CachingRepository → LoggingRepository
- In .NET: Stream is a decorator (GZipStream wraps FileStream)
- Interview: 'How do MediatR pipeline behaviours use the

Strategy

Define a family of algorithms, encapsulate each one, and make them interchangeable. Vary the algorithm independently.

- IDiscountStrategy with LoyaltyDiscount, BulkDiscount, VipDiscount
- Inject via DI — swap strategy without changing OrderService
- Strategy = OCP in practice: new discount type = new class only
- IPaymentGateway: StripeGateway, PayFastGateway — swap per region
- Feature flags: inject different strategy based on flag value
- Interview: 'How does Strategy implement the Open/Closed Principle?'

Observer

When one object changes, all its dependents are notified automatically. C# events and MediatR INotification are Observer built in.

- C# events: orderPlaced += (s,e) => SendEmail(e)
- Domain events: Order raises OrderPlacedEvent — multiple handlers subscribe
- MediatR INotification + INotificationHandler — ordered, async observer
- Service Bus = Observer at infrastructure level — topic + multiple subscribers
- Benefit: decouples the publisher from the subscribers — neither knows about each other
- Interview: 'Difference between domain events and integration events?'

Mediator & Command

Mediator: objects communicate through it, not directly. Command: encapsulate a request as an object.

- MediatR IS the Mediator — sender.Send(command) — sender doesn't know the handler
- Command: PlaceOrderCommand encapsulates all data to place an order
- Commands can be queued, retried, logged, undone — because they're objects
- Enables: audit log, undo/redo, event sourcing, CQRS
- Mediator benefit: add a new handler without changing the sender
- Interview: 'How does MediatR implement both Mediator and Command patterns?'

CLEAN ARCHITECTURE

Clean Architecture: layers, rules & folder structure

The dependency rule: source code dependencies only point INWARD. Domain knows nothing about SQL, HTTP, or Azure. Nothing should force a change to the

Domain

Pure C#. Zero NuGet dependencies. Business rules live here and NOWHERE else.

- Entities: Order, Customer — have identity (Id), mutable state
- Value objects: Money, Address — immutable, equality by value
- Domain events: OrderPlaced, PaymentFailed — facts that happened
- Interfaces: IOrderRepository — defined here, implemented in Infrastructure
- Aggregates: consistency boundary — always load/save the Aggregate Root

Application

Orchestrates the Domain. Use cases (what the app can DO). Depends only on Domain.

- CQRS: Commands (write) + Queries (read)
- Command/query handlers
- Pipeline behaviours: validation, logging, caching
- Application services: coordinate multiple domain operations
- DTOs and mapping (AutoMapper or manual)
- Interfaces for infrastructure: IEmailService (defined here)

Infrastructure

Implements interfaces. Can be swapped (SQL → Cosmos) without touching Domain or Application.

- SqlOrderRepository implements IOrderRepository
- AppDbContext: EF Core config + migrations
- Azure SDK clients: Service Bus, Blob, Key Vault
- External API clients: payment gateways, email providers
- Depends on: Domain + Application (for interfaces it implements)

API / Presentation

Entry point. HTTP → Command/Query. No business logic here.

- Minimal API endpoints or ASP.NET Core controllers
- Middleware: auth, error handling, request logging
- DI registration in Program.cs
- OpenAPI / Swagger documentation
- Maps: HTTP req → command → MediatR → HTTP response

Folder structure

```
MyApp.sln
├─ src/
│  ├─ MyApp.Domain/
│  │  ├─ Entities/
│  │  ├─ Events/
│  │  └─ ValueObjects/
│  ├─ MyApp.Application/
│  │  ├─ Orders/
│  │  │  ├─ Commands/
│  │  │  └─ Queries/
│  │  └─ Behaviours/
│  ├─ MyApp.Infrastructure/
│  │  ├─ Persistence/
│  │  └─ Services/
│  └─ MyApp.API/
│     ├─ Endpoints/
│     └─ Middleware/
└─ tests/
   ├─ Unit/
   ├─ Integration/
   └─ Architecture/
```

Interview Q: 'How do you enforce Clean Architecture rules in CI?'

A: NetArchTest — assert that Domain has no dependency on Infrastructure. Runs as a unit test in CI pipeline, fails the build if violated.

Best resource: 'Clean Architecture' — Robert C. Martin (Uncle Bob)

CQRS + MEDIATR

CQRS and MediatR: commands, queries & pipeline behaviours

CQRS separates reads from writes. A Command changes state and returns nothing (or just an ID). A Query reads state and NEVER changes it. MediatR wires

Command (write side)

Commands represent intent: PlaceOrder, CancelOrder. They validate, execute business logic, and raise domain events.

```
// Command — intent to change state
public record PlaceOrderCommand(
  int CustomerId, List<OrderItemDto> Items
) : IRequest<Result<int>>;

// Handler — all logic for this use case
public class PlaceOrderHandler(
  IOrderRepository repo, IUnitOfWork uow,
  IPublisher publisher
) : IRequestHandler<PlaceOrderCommand, Result<int>>
{
  public async Task<Result<int>> Handle(
    PlaceOrderCommand cmd, CancellationToken ct)  {
    var order = Order.Create(cmd.CustomerId,
cmd.Items);
```

- Each handler is independent — one file, one responsibility (SRP)
- Commands should validate via pipeline behaviour BEFORE the handler runs
- Result<T> pattern: return success/failure without throwing for flow control
- Domain events raised INSIDE the handler — infrastructure handles them outside
- Interview: 'Why should commands not return the full entity?'

Query + Pipeline Behaviour

```
// Query — reads state, NEVER modifies
public record GetOrdersQuery(int CustomerId, int
Page)
  : IRequest<PagedResult<OrderDto>>;

public class GetOrdersHandler(AppDbContext ctx)
  : IRequestHandler<GetOrdersQuery,
PagedResult<OrderDto>> {
  public async Task<PagedResult<OrderDto>> Handle(
    GetOrdersQuery q, CancellationToken ct) =>
```

Validation Pipeline Behaviour

```
// Runs BEFORE every handler automatically
public class ValidationBehaviour<TReq,TRes>(
  IEnumerable<IValidator<TReq>> validators
) : IPipelineBehavior<TReq,TRes> {
  public async Task<TRes> Handle(TReq req,
    RequestHandlerDelegate<TRes> next,
CancellationToken ct) {
```

- Behaviours are Decorators — each wraps the next in the chain
- Register: services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehaviour<,>))
- Order matters: Logging → Validation → Handler
- Interview: 'How do pipeline behaviours implement cross-cutting concerns?'

DOMAIN-DRIVEN DESIGN

Domain-driven design: ubiquitous language to bounded contexts

DDD is both a language (to communicate with the business) and a set of patterns (to model complex business logic in code). When the code speaks the same

Building blocks

Use DDD vocabulary when talking to business stakeholders and writing code — bridge the communication gap.

- Entity: has identity (OrderId) — same object across time, mutable state
- Value Object: no identity (Money, Address) — equality by value, IMMUTABLE
- Aggregate: consistency boundary — always access through the Aggregate Root
- Aggregate Root: the only entry point (Order, not OrderItem directly)
- Domain Event: a fact that happened (OrderPlaced, PaymentReceived)
- Repository: one per Aggregate Root — IOrderRepository

Bounded contexts

'Order' in Billing means Invoice. 'Order' in Fulfilment means Shipment. Different models for the same word. Each context has its own code.

- Bounded Context = a subsystem with its own domain model and language
- Context map: how contexts relate — Shared Kernel, Customer/Supplier, Conformist
- Anti-Corruption Layer (ACL): translate between contexts — don't pollute your model
- One bounded context = one microservice (when you eventually split)
- Ubiquitous language: code uses exactly the same words as the business uses
- If you find yourself adding 'type' flags, you may need two bounded contexts

Aggregate design example

```
// Order is the Aggregate Root — it owns its Items
public class Order : AggregateRoot {
  private readonly List<OrderItem> _items = [];
  public IReadOnlyList<OrderItem> Items => _items.AsReadOnly(); // protected

  public static Order Create(int customerId, List<OrderItemDto> items) {
    var order = new Order(customerId);
    items.ForEach(i => order.AddItem(i)); // domain method enforces invariants
    order.RaiseDomainEvent(new OrderCreatedEvent(order.Id)); // domain event
    return order;
```

EF CORE + DAPPER

EF Core & Dapper: when to use each

EF Core for writes and complex domain queries. Dapper for complex read queries and reporting where raw SQL is cleaner. Use both in the same project.

EF Core essentials

```
// DbContext — Unit of Work + gateway to DB
public class AppDbContext(DbContextOptions<AppDbContext> opts,
  ITenantContext tenant) : DbContext(opts) {
  public DbSet<Order> Orders => Set<Order>();

  protected override void OnModelCreating(ModelBuilder mb) {
    mb.ApplyConfigurationsFromAssembly(GetType().Assembly);
    // Global filter — EVERY query adds WHERE TenantId = @id
    mb.Entity<Order>().HasQueryFilter(o => o.TenantId ==
tenant.Id);
  }

  public override async Task<int>
SaveChangesAsync(CancellationToken ct=default){
    foreach (var e in ChangeTracker.Entries<BaseEntity>())
      if (e.State == EntityState.Added) e.Entity.CreatedAt =
DateTime.UtcNow;
    return await base.SaveChangesAsync(ct);
  }
```

EF Core vs Dapper

	EF Core	Dapper
Best for	Writes + complex domain queries	Read queries + reporting SQL
Mapping	Automatic + configurable	Manual or Dapper conventions
SQL control	Generated SQL (inspectable)	You write the SQL
Performance	Good with AsNoTracking()	Slightly faster for simple reads

Common EF Core pitfalls

- N+1: foreach order → load customer = N+1 queries. Fix: .Include(o => o.Customer)
- Client eval: Where(o => MyMethod(o)) loads ALL rows first
- SaveChanges inside loop: batch all changes, save once at the end
- Long-lived DbContext: always Scoped (per request), never Singleton
- Missing index: EF doesn't add indexes automatically — add via fluent config
- Over-fetching: Select(o => new Dto { … }) only loads needed columns
- Soft delete: HasQueryFilter(o => !o.IsDeleted) — never leaks deleted records
- Concurrency: use RowVersion + DbUpdateConcurrencyException handler

TESTING STRATEGY

Testing: pyramid, tools & architecture tests

The testing pyramid: many fast unit tests, fewer integration tests, minimal E2E tests. Architecture tests enforce structural rules at build time.

Unit tests (70%)

Fast, isolated, mock everything external. One failing test = one problem. Run in milliseconds.

- xUnit (preferred) / NUnit / MSTest — all fine, xUnit is most modern
- NSubstitute (preferred) or Moq for mocking
- FluentAssertions: result.Should().BeTrue() — readable failures
- Naming: MethodName_Scenario_ExpectedBehaviour
- Test the HANDLER not the controller — business logic is in the handler
- Each test: Arrange → Act → Assert
- No DB, no HTTP, no files — inject fakes for everything external

Integration tests (20%)

Test real interactions — real DB, real HTTP. Use Testcontainers to spin up a real SQL Server in Docker.

- WebApplicationFactory<Program>: spin up real ASP.NET Core in memory
- Testcontainers: spin up real SQL Server / Redis / Service Bus in Docker
- Override DI registrations: use test DB, not prod DB
- Test the full pipeline: HTTP request → handler → DB → HTTP response
- Slower (seconds) — run in CI, not on every save
- Respawn (Jimmy Bogard): clean DB between tests without recreating it
- Contract tests: Pact.NET — verify API consumer and provider

Architecture tests (∞ value, 0 maintenance)

Assert structural rules as unit tests. NetArchTest runs in milliseconds and fails the build if architecture is violated.

- Domain must not depend on Infrastructure — verified every build
- All handlers must be in Application layer
- Entities must not have public setters
- NetArchTest.Rules NuGet — 5 lines to enforce a rule
- Run in CI — prevents architecture drift silently accumulating
- Pair with Roslyn Analyser for compile-time enforcement
- Interview: 'How do you enforce Clean Architecture in a team?'

Test best practices

Good tests are an asset. Bad tests are a maintenance burden.

- Coverage target: 80% line coverage as a CI gate — not a goal
- Mutation testing: Stryker.NET verifies tests actually catch bugs
- Don't test implementation detail — test observable behaviour
- Test the unhappy path too: null inputs, boundary values, domain exceptions
- Fake vs Mock vs Stub: Fake has logic; Mock verifies calls; Stub returns canned data
- Builder pattern for test data: new OrderBuilder().WithOverduePayment().Build()
- Interview: 'What is the difference between a Mock and a Stub?'

DI & OPTIONS PATTERN

Dependency injection lifetimes & the Options pattern

DI is built into ASP.NET Core. Choosing the wrong lifetime causes stale data, memory leaks, or threading bugs — the captive dependency problem is the most

Lifetimes explained

```
// SINGLETON — one instance for app lifetime
builder.Services.AddSingleton<ICache,
RedisCache>();
// ✅ Use for: config, caches, HTTP clients,
connection pools
// ❌ DANGER: must be thread-safe if it holds state

// SCOPED — one per HTTP request (or per scope)
builder.Services.AddScoped<IOrderRepository,
SqlOrderRepository>();
builder.Services.AddScoped<AppDbContext>();
// ✅ Use for: DbContext, repositories, anything
per-request
// ❌ NEVER inject into Singleton — captive
dependency!

// TRANSIENT — new instance every injection
builder.Services.AddTransient<IEmailSender,
SmtpEmailSender>();
// ✅ Use for: lightweight, stateless services
// ⚠ Avoid for DbContext — too many instances

// CLASSIC BUG: Singleton holds reference to Scoped
public class CacheWarmer(AppDbContext ctx)
Singleton
  ConfigService(AppDbContext ctx) {}    //
AppDbContext is Scoped
  // First request's DbContext is held forever →
```

- DbContext MUST be Scoped — one per request, tracks changes
- HttpClient: use IHttpClientFactory (Singleton) — avoids socket exhaustion
- IServiceProvider.CreateScope() for background services needing Scoped services
- Test: IServiceScopeFactory.CreateScope().ServiceProvider.GetRequiredService<T>()
- Interview: 'Explain the captive dependency problem with a concrete example'

Options pattern — strongly-typed config

```
// appsettings.json
{ "Email": { "SmtpHost": "smtp.sg.com", "Port":
587, "From": "noreply@co.za" } }

// Strongly-typed options
public class EmailOptions {
  public const string Section = "Email";
  [Required] public string SmtpHost { get; set; } =
"";
  [Range(1,65535)] public int Port { get; set; }
  [Required][EmailAddress] public string From
{ get; set; } = "";
}

// Register with validation on startup
builder.Services.AddOptions<EmailOptions>()
  .Bind(builder.Configuration.GetSection(EmailOptions.Section))
  .ValidateDataAnnotations()
  .ValidateOnStart(); // fail fast at app start,
not first use

// Use
public class EmailService(IOptions<EmailOptions>
opts) {
  void Send() {
    var cfg = opts.Value; // strongly typed,
validated
    // cfg.SmtpHost, cfg.Port, cfg.From
```

3 Options interfaces

- IOptions<T>: snapshot at app start — simplest, no live reload
- IOptionsSnapshot<T>: refreshed per request — good for per-tenant config
- IOptionsMonitor<T>: live reload without restart — good for feature flags
- Prefer ValidateOnStart() — catch config errors immediately at deployment
- Never use config["key"] strings — always a typed options class
- Interview: 'Why use IOptions over IConfiguration directly?'

OBSERVABILITY & RESILIENCE

Observability: logging, tracing, metrics & Polly resilience

You cannot fix what you cannot see. Observability = structured logging + distributed tracing + metrics. Polly = resilience against failures in downstream services.

Structured logging with Serilog

Structured logging embeds searchable properties in log messages. Unstructured logs are unsearchable strings — useless at scale.

- _logger.LogInformation("Order {OrderId} placed by {CustomerId}", id, custId)
- The {} syntax embeds searchable properties — query in Log Analytics
- Serilog.AspNetCore: request logging middleware in one line
- Log levels: Trace < Debug < Information < Warning < Error < Critical
- NEVER log: passwords, JWT tokens, PII, connection strings
- LogContext.PushProperty('TenantId', id) — ambient enrichment for all logs in scope
- Interview: 'What is the difference between structured and

Distributed tracing (OpenTelemetry)

One user request may cross 5 services. Distributed tracing links all the spans into one trace so you can see the full picture.

- Activity API: automatic for HttpClient, EF Core, Service Bus, ASP.NET Core
- W3C traceparent header: propagated across service boundaries automatically
- App Insights: end-to-end transaction map — see all hops in one view
- Custom span: using var activity = ActivitySource.StartActivity('ProcessOrder')
- Correlation ID: inject into all log messages via enricher
- Export to: App Insights / Jaeger / Zipkin / OTLP endpoint — vendor-neutral

Polly v8 resilience pipelines

Polly adds resilience to any call that can fail: HTTP, DB, Service Bus. Configure once, apply everywhere via DI.

- Retry: 3 attempts with exponential backoff + jitter
- Circuit breaker: after 5 failures, open for 30s — fast fail, spare the downstream
- Timeout: fail after 2s — don't hang waiting for a dead service
- Hedge: send duplicate request if first is slow — race to first response
- Bulkhead: limit concurrent calls to downstream — prevent cascade failure
- services.AddResiliencePipeline('downstream', b => b.AddRetry().AddCircuitBreaker())
- Interview: 'What is the difference between Retry and Circuit Breaker?'

Health checks & metrics

Health checks tell Kubernetes and App Service whether your app is ready to serve traffic. Metrics drive alerts.

- app.MapHealthChecks('/health/ready'): is DB available? Cache up? Deps healthy?
- app.MapHealthChecks('/health/live'): is the process alive? (simple 200 response)
- AKS readiness probe: won't route traffic until /health/ready returns 200
- Custom metric: Meter.CreateCounter<long>('orders.placed') — push to App Insights
- Alert on: p99 latency > 2s, error rate > 1%, GC Gen2 collections high
- Prometheus + Grafana: industry-standard metrics stack — Azure Monitor Workspace
- Interview: 'What is the difference between a liveness probe and

API DESIGN

REST API design: principles, versioning & error contracts

Good API design is a contract with your consumers. Breaking it costs trust and forces them to change their code. Design contracts first, implement second.

REST principles

```
// ✅ Resource-oriented nouns — not verbs
GET    /api/v1/orders/{id}          // read one
GET    /api/v1/orders?status=active // filter
POST   /api/v1/orders               // create (returns 201 +
Location header)
PUT    /api/v1/orders/{id}          // full replace
PATCH  /api/v1/orders/{id}          // partial update
DELETE /api/v1/orders/{id}          // delete

// ✅ Consistent error shape (RFC 7807 Problem Details)
{ "type": "https://api.myco.com/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "CustomerId is required",
  "traceId": "00-abc123-def456-00" }
```

HTTP status codes to know

```
200: OK — success
201: Created — POST succeeded, return Location
204: No Content — DELETE/PUT succeeded
400: Bad Request — client sent invalid data
401: Unauthorized — not authenticated
403: Forbidden — authenticated but not allowed
404: Not Found — resource doesn't exist
409: Conflict — duplicate, optimistic concurrency
422: Unprocessable — validation failed
429: Too Many Requests — rate limited
500: Internal Server Error — server bug
```

Versioning & design rules

- URL versioning: /api/v1/ — simplest, most discoverable
- Header: api-version: 2.0 — cleaner URLs
- Asp.Versioning.Http NuGet — version support in .NET
- Deprecated version: Sunset: Sat, 01 Jan 2026 00:00:00 GMT header
- OpenAPI/Swagger: Swashbuckle or Scalar — auto-generated docs
- Pagination: cursor-based (after=lastId) or skip+take with totalCount
- HATEOAS: include related action links in response (advanced)
- gRPC: binary protocol, strongly typed (protobuf), faster than REST — use for internal service-to-service calls
- GraphQL: client specifies shape — good for flexible frontends
- Interview: 'When would you use gRPC over REST?'
- REST interview: 'What makes an API truly RESTful?'

DevOps & Azure Cloud

Git · CI/CD · Docker · Kubernetes · Security · Redis · Service Bus

GIT & SOURCE CONTROL

Git: branching, commits & enterprise practices

Git is more than version control — it is the gating mechanism that ensures quality before code reaches production. Branch protection and conventional commits

Branching strategy

Trunk-based development (short-lived branches, merge daily) is preferred over long-lived GitFlow for CI/CD teams.

- main = production state at all times — never push directly
- feature/<ticket-id> — short-lived (days, not weeks)
- PR → code review → CI green → squash merge → main
- Branch protection: require 1 approval + all CI checks pass
- Conventional commits: feat: / fix: / chore: / refactor: / docs:
- Tags for releases: v1.2.3 — auto-generate release notes
- GitFlow: use for OSS libraries; avoid for microservices teams

Secrets & security

A secret committed to Git is compromised forever — rotate immediately, even if you delete it in the next commit.

- NEVER commit: connection strings, API keys, passwords, certificates
- Local dev: dotnet user-secrets — stored outside the repo
- CI/CD: Key Vault or pipeline secret variables — never hardcoded
- Pre-commit hook: git-secrets blocks known secret patterns
- If committed: rotate the secret immediately (not just delete the commit)
- GitHub/Azure DevOps secret scanning: alerts on detected secrets in pushes

Pull requests & code reviews

PRs are where knowledge spreads and quality is gated. Good PR descriptions are worth more than perfect code.

- PR description: what changed, why, how to test, risk level
- PR size: <400 lines changed — large PRs get rubber-stamped
- Squash merge: keeps main history clean — one commit per feature
- Link to ticket: traceability from code to requirement
- Review checklist: tests present? SOLID respected? Security considered?
- Reviewers: at least one senior developer for critical paths
- Interview: 'What makes a good pull request description?'

Release management

Semantic versioning communicates the impact of a change. Tags enable rollback to any version.

- semver: MAJOR.MINOR.PATCH — breaking.feature.bugfix
- MAJOR: breaking change; MINOR: new feature (backwards compatible); PATCH: bugfix
- Release notes: auto-generated from conventional commit messages
- Changelog: CHANGELOG.md — maintained by tooling (semantic-release)
- Hotfix: branch from main, fix, merge back with PATCH bump
- Feature flags: deploy code without activating it — decouple deploy from release
- Interview: 'When would you use a feature flag instead of a branch?'

CI/CD PIPELINES

CI/CD: from commit to production, safely

CI proves the code works. CD proves it can be deployed. Both together mean every merge to main is releasable. This is the gold standard for enterprise .NET

CI pipeline stages

Fail fast: cheapest checks first. A typo fails in 30 seconds, not after a 10-minute Docker build.

1	**Restore & build**	dotnet restore → dotnet build — catch compile errors
2	**Unit tests**	dotnet test — fast, no DB, must be >80% coverage
3	**Code quality**	SonarQube SAST, code smell detection
4	**Dependency scan**	OWASP NuGet dependency-check for CVEs
5	**Docker build**	Multi-stage Dockerfile → push to ACR

```
# GitHub Actions example
on: [push]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: dotnet restore && dotnet build --no-
```

CD deployment strategies

Goal: deploy without users noticing. The three strategies differ in risk, complexity and rollback speed

Rolling update

Replace instances one at a time. Brief mixed-version period. Simplest to implement.

Rollback: re-deploy previous image tag

Blue-green

New version runs alongside old. Switch traffic all at once. Easy rollback: flip switch.

Rollback: Flip traffic back to blue

Canary release

5% of traffic to new version. Monitor errors. Increase if healthy over 30 minutes.

Rollback: Route 0% to canary, investigate

Environments & approval gates

- dev: auto-deploy on every merge to main — no approval
- staging: auto-deploy + automatory manual approval gate
- production: manual approval + change record + scheduled window
- Rollback: re-deploy previous Docker image tag — seconds not minutes
- Health check: /health/ready must return 200 within 60s of deploy
- Interview: 'What is the difference between CI and CD?'
- Interview: 'Describe blue-green deployment and its benefits'

DOCKER & KUBERNETES

Docker containers & AKS Kubernetes essentials

Docker packages your app and its dependencies into a portable image. Kubernetes runs many containers reliably across many servers, healing failures

Multi-stage Dockerfile

Multi-stage: build in SDK image (large), copy artifacts to runtime image (small). Production image is minimal — faster pull, smaller attack surface

```
# Stage 1: build (large SDK image — not shipped)
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY ["MyApp.API/MyApp.API.csproj", "MyApp.API/"]
RUN dotnet restore "MyApp.API/MyApp.API.csproj"
COPY . .
RUN dotnet publish "MyApp.API/MyApp.API.csproj" -c
Release -o /app

# Stage 2: runtime (small runtime image — this is
what ships)
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS final
WORKDIR /app
COPY --from=build /app .
# Run as non-root user — security best practice
USER $APP_UID
ENTRYPOINT ["dotnet", "MyApp.API.dll"]
```

- Final image is ~200MB vs SDK image ~900MB — use multi-stage always
- docker-compose.yml: local dev with real dependencies (SQL Server, Redis)
- HEALTHCHECK in Dockerfile: Docker knows if container is unhealthy
- .dockerignore: exclude bin/, obj/, .git — smaller build context
- Trivy or Grype: scan image for CVEs in CI pipeline
- Never run as root in production — USER instruction in Dockerfile
- Interview: 'What is a multi-stage Docker build and why use it?'

AKS Kubernetes concepts

You describe WHAT you want. Kubernetes makes it happen and keeps it that way — healing failures, scaling pods, routing traffic.

Pod	*Smallest unit — one or more containers, shared network + storage*
Deployment	*Desired state — '3 replicas of image:v2.1'. Self-healing*
Service	*Stable DNS + IP for a pod set — pods come and go, Service is stable*
Ingress	*HTTP routing — routes /api/orders → orders-service + TLS termination*
ConfigMap	*Non-secret config injected as env vars or files*

```
# Deployment manifest (key fields)
spec:
  replicas: 3
  strategy: { type: RollingUpdate }
  template:
    spec:
      containers:
        - name: api
          image: myacr.azurecr.io/myapp:v2.1
```

SECURITY & IDENTITY

Security: JWT anatomy, OAuth 2.0, Azure AD & Key Vault

Every request must prove identity (authentication) and what it's allowed to do (authorisation). JWT is the token format; OAuth 2.0 is the flow; Azure AD is the

JWT token anatomy

Header.Payload.Signature — three base64 JSON objects. Signature proves it wasn't tampered with. Payload contains claims — facts about

```
// Decoded JWT payload
{
  "sub":  "user-guid",            // who this token
is about
  "iss":
"https://login.microsoftonline.com/{tenant}", //
issuer
  "aud":  "api://my-app-client-id",  // intended
recipient
  "exp":  1749600000,             // expiry (Unix
timestamp)
  "iat":  1749596400,             // issued at
  "roles": ["Admin","ReadOnly"],  // RBAC roles
  "tid":  "tenant-id-guid",       // Azure AD
tenant
```

Validate in .NET

```
builder.Services
  .AddAuthentication(JwtBearerDefaults.Authenticati
onScheme)
  .AddJwtBearer(o => {
    o.Authority =
"https://login.microsoftonline.com/{tid}";
    o.Audience  = "api://my-app-client-id";
    // Azure AD's public key fetched automatically
from Authority
  });
```

OAuth 2.0 flows

OAuth 2.0 is the protocol for getting access tokens. Choose the flow based on whether a human is present

Auth Code + PKCE

Web, SPA, mobile — user logs in

Redirect → user authenticates → auth code → exchange for token

Client Credentials

Service-to-service — no human user

Service sends clientId + secret → gets token → calls API

On-Behalf-Of (OBO)

API calling another API on user's behalf

API receives user token → exchanges for token for downstream

Key Vault + Managed Identity

```
// Managed Identity — no credentials stored
anywhere
var client = new SecretClient(
  new Uri("https://myvault.vault.azure.net"),
  new DefaultAzureCredential()); // uses MI
automatically
// Never store clientId/secret in config for Azure
```

AZURE INFRASTRUCTURE

Azure infrastructure: full service map with guidance

Azure has hundreds of services. Knowing WHEN to use each — and when NOT to — is what separates a developer from a cloud architect.

Compute — choose your host

The choice determines your operational overhead. More control = more responsibility.

- App Service: managed PaaS — Microsoft handles OS patches, scaling. Use for: simple APIs, easy to operate
- Container Apps: serverless containers, scale to zero. Use for: event-driven, variable load workloads
- AKS: full Kubernetes control. Use for: complex multi-service, 24/7 high-scale workloads
- Azure Functions: serverless, event-triggered. Use for: background jobs, scheduled tasks, lightweight APIs
- Decision: App Service first → Container Apps → AKS (add complexity only when justified)

Networking — secure the perimeter

Every production Azure system should have a VNet. Expose as little as possible to the internet.

- VNet: isolate resources from internet — always use in production
- NSG: Network Security Group — firewall rules per subnet (stateful)
- Private Endpoint: PaaS service (SQL, Storage, Key Vault) accessible only from VNet
- Application Gateway (L7 load balancer + WAF): WAF blocks OWASP top-10 attacks
- API Management: single entry point, policies, rate limiting, dev portal
- Front Door: global CDN + WAF + anycast routing — for globally

Messaging — async communication

Async messaging decouples services. Producer doesn't wait for consumer — if slow or temporarily down.

- Service Bus Queues: one message → one consumer (point-to-point)
- Service Bus Topics: one message → many consumers (fan-out / pub-sub)
- Event Grid: reactive events for Azure resource events (blob uploaded, etc.)
- Event Hubs: high-throughput streaming (millions of events/sec) — analytics pipeline
- Dead-letter queue: failed messages for inspection and replay
- Use Service Bus for: business events between services
- Use Event Hubs for: telemetry, logs, click streams at scale

Data — choose the right store

There is no single best database. Use the right store for the access pattern.

- Azure SQL: relational, ACID, perfect for structured business data — your default
- Cosmos DB: globally distributed NoSQL, multi-model. Use when: global scale, schema flexibility
- Redis Cache: in-memory key-value — sub-millisecond reads. Use for: caching, sessions, rate limiting
- Blob Storage: files, images, documents, backups — cheap object storage
- Table Storage: simple NoSQL, very cheap. Use for: logs, diagnostics, simple key-value
- Interview: 'When would you choose Cosmos DB over Azure SQL?'

REDIS & SERVICE BUS

Redis & Service Bus: patterns, code & pitfalls

Redis = fast memory cache. Service Bus = reliable async messaging. Together they are the backbone of every scalable .NET Azure system.

Redis: cache-aside pattern

Check cache first. Miss → go to DB → store in cache. Always set TTL. Always invalidate on write.

```
public async Task<CustomerDto?> GetAsync(int id) {
  var key = $"customer:{id}";

  // 1. Check cache
  var cached = await _cache.GetStringAsync(key,
ct);
  if (cached != null)
    return
JsonSerializer.Deserialize<CustomerDto>(cached)!;

  // 2. Miss — go to DB
  var customer = await _repo.GetByIdAsync(id, ct);
  if (customer is null) return null;

  // 3. Store with TTL — ALWAYS set expiry
  await _cache.SetStringAsync(key,
    JsonSerializer.Serialize(customer),
    new DistributedCacheEntryOptions {
      AbsoluteExpirationRelativeToNow =
TimeSpan.FromMinutes(5)
    }, ct);
  return customer;
}

// On update — invalidate cache
await _cache.RemoveAsync($"customer:{id}", ct);
```

- Unbounded cache (no TTL) = memory leak — always set TTL
- Stale-while-revalidate: return stale + trigger background refresh
- Cache stampede: all threads miss simultaneously → Redlock distributed lock
- Redis also does: rate limiting, pub/sub, distributed sessions, leaderboards
- Never cache sensitive PII without encryption at rest
- Interview: 'What is the cache stampede problem and how do you fix it?'

Service Bus: patterns

Without Service Bus: Email service down = email lost. With Service Bus: message waits in queue until Email service recovers.

```
// Publisher — OrderService sends event
await _sender.SendMessageAsync(new
ServiceBusMessage(
  JsonSerializer.SerializeToUtf8Bytes(
    new OrderPlacedEvent(order.Id, order.Total)
  )) {
  MessageId = Guid.NewGuid().ToString(), //
idempotency
  Subject   = "OrderPlaced",
  SessionId = order.CustomerId.ToString() //
ordered delivery
});

// Consumer — EmailService handles event
async Task Handle(ProcessMessageEventArgs args) {
  var evt =
JsonSerializer.Deserialize<OrderPlacedEvent>(
    args.Message.Body);
  await _email.SendConfirmationAsync(evt!);
  await args.CompleteMessageAsync(args.Message); //
ACK
  // If crash here: message stays in queue for
retry
  // After maxDeliveryCount: moved to Dead-Letter
Queue
}
```

- Queue: competing consumers — one message to one consumer (load balancing)
- Topic + subscriptions: fan-out — Email, Inventory, Analytics all get OrderPlaced
- Dead-letter queue (DLQ): inspect and replay failed messages — production essential
- Message lock: consumer holds lease; must Complete before lock expires
- Idempotent consumer: store MessageId in DB to detect duplicates on retry
- Session: guaranteed order per SessionId (per customer, per tenant)
- Interview: 'How do you make a Service Bus consumer

System Architecture

Design · DDD · Multi-tenancy · Microservices · Distributed Systems

SYSTEM DESIGN

System design: trade-offs, NFRs & architect thinking

Architects don't choose the best technology. They choose the best technology for these specific constraints: budget, team, compliance, scale, and timeline.

CAP theorem & consistency

In distributed systems, you can only guarantee 2 of 3: Consistency, Availability, Partition Tolerance. Partition tolerance is always required — so the real choice is C vs A.

- Consistency: every read gets the latest write (SQL, your Transfer Agency)
- Availability: every request gets a response, possibly stale (Cosmos AP mode)
- Strong consistency: ACID, 2PC distributed transactions — simple but slow
- Eventual consistency: Service Bus events, CQRS read models — fast but stale temporarily
- BASE: Basically Available, Soft state, Eventually consistent — NoSQL default
- Interview: 'How does CAP theorem affect your service bus

Non-functional requirements (NFRs)

NFRs define HOW the system behaves, not WHAT it does. Missing NFRs lead to systems that work in demos but fail in production.

- Latency: p95 < 200ms, p99 < 500ms at the API Gateway — measure at the edge
- Availability: 99.9% = 8.7 hrs downtime/yr; 99.99% = 52 min/yr
- RTO: how fast to restore after disaster? (hours? minutes? seconds?)
- RPO: how much data can be lost? (0 for financial — synchronous replication)
- Throughput: 1,000 req/s? 100,000 req/s? Determines DB + cache + pod count
- Compliance: GDPR (EU data), POPIA (ZA data protection), PCI-

Monolith vs microservices

Start with a modular monolith. Split only when you have concrete evidence: different scaling needs, different teams, different release frequency.

- Modular monolith: one deployable unit, logical module boundaries, shared DB
- Microservices: independent deployable, independent DB, communicate via events/HTTP
- Distributed monolith: worst of both worlds — split but tightly coupled. AVOID
- Split signal: different teams need independent releases OR one service needs 10× scale
- Each service owns its own database — no shared DB between services
- Interview: 'When would you advise AGAINST microservices?'

Distributed system patterns

Every pattern solves a specific failure mode. Know the problem first, then apply the pattern.

- Saga: distributed transaction across services — choreography (events) or orchestration (coordinator)
- Outbox pattern: write to DB + outbox table in one transaction — publish events reliably
- Circuit breaker: stop calling failing service — fast fail, allow recovery
- Retry + exponential backoff: handle transient failures without hammering the downstream
- Bulkhead: limit concurrent calls — prevent one slow service taking down everything
- Event sourcing: store events not current state — full audit trail, rebuild any point in time

MULTI-TENANCY

Multi-tenancy: silo, pool & bridge patterns

A tenant is an organisation using your SaaS. Your Transfer Agency almost certainly uses the silo model. SaaS products use pool or bridge. Know the trade-offs

Silo — DB per tenant

Pros

- Maximum data isolation — breach affects one tenant
- Easy compliance: GDPR, POPIA, PCI-DSS per tenant
- Independent backup, restore, migration per tenant
- Simple: no TenantId columns, no query filters needed

Cons

- Highest infra cost — N databases = N connection pools
- Cross-tenant reporting is hard
- Schema migrations run N times
- Connection management grows with tenants

Use when: Transfer Agency, healthcare, legal, high-compliance SaaS

Pool — Shared DB + TenantId

Pros

- Lowest cost — dense packing, one database
- Single migration to maintain
- Simple cross-tenant analytics

Cons

- Missing WHERE TenantId = data leak (CRITICAL risk)
- Noisy neighbour: one tenant's query slows all
- RLS required for safety — adds DB complexity

Use when: SMB SaaS, internal tooling, low-sensitivity data

Bridge — Shared compute, separate DBs

Pros

- DB-level isolation without RLS risk
- Cost-efficient shared compute
- Balance of cost vs compliance

Cons

- Connection string management grows
- Connection pool overhead per tenant DB

Use when: Most common modern SaaS — best balance

```
// Pool model: EF Core global query filter — automatically adds WHERE TenantId = @id to EVERY query
```

Career Paths & Intent Architect

DevOps · Solutions Architect · Enterprise Architect · Tool Builder

CAREER ROLES

The 6 career paths: roles, responsibilities & certifications

Each role builds on the previous. The skills compound. DevOps and Solutions Architect can be pursued in parallel from senior developer level.

Senior .NET Developer

Cert: AZ-204

- Own a feature area end-to-end — design, build, test, deploy
- Mentor junior developers — code reviews, pair programming
- Contribute to architecture decisions with evidence
- Deep CQRS, Clean Architecture, DDD expertise
- Performance profiling and optimisation

DevOps Engineer

Cert: AZ-400

- Own CI/CD pipelines from YAML to deployment
- Infrastructure as Code: Bicep or Terraform
- Kubernetes cluster operations and upgrades
- Observability: dashboards, alerts, on-call
- Security scanning in pipelines

Solutions Architect

Cert: AZ-305

- Design the technical blueprint for a system or project
- Produce C4 models, ADRs, API contracts, event schemas
- Define NFRs: latency, availability, RTO, RPO
- Present trade-offs to technical and non-technical stakeholders
- Threat modelling (STRIDE), security architecture

Cloud Architect

Cert: AZ-305 + specialisation

- Multi-region, highly available Azure architecture
- Landing zones, hub-spoke networking, private connectivity
- Azure Well-Architected Framework: 5 pillars
- Cost optimisation: right-sizing, reserved instances
- FinOps: tag resources, showback/chargeback

Enterprise Architect

Cert: TOGAF 10

- Organisation-wide technology strategy and roadmap
- Governance: standards, patterns, vendor selection
- Business capability mapping (ArchiMate)
- Portfolio management: what to build vs buy vs retire
- Board-level technology investment advice

Tool Builder

Cert: Roslyn + architecture depth

- Build Roslyn analysers and source generators
- Develop Intent Architect modules
- Create internal developer platforms (IDPs)
- Design DSLs and code generation pipelines
- Sell tools to other developers and teams

Intent Architect: architecture-centric AI code automation

South African-built .NET platform. Visual design → deterministic code generation → AI agents constrained by your architecture. 4×–10× faster development.

What it does

You describe your system visually. IA generates code that 100% conforms to your architecture. When design changes, IA regenerates — automatically.

- Visual designer: define entities, services, events — living blueprints
- Architecture modules: Clean Arch, CQRS, EF Core, FastEndpoints, gRPC, React, Angular, 100+
- Code generation: every file conforms to pattern — zero drift between developers
- AI agents: Claude/Copilot code within IA's architecture guardrails
- Regeneration: change entity field → IA updates all affected files instantly

Problems it solves

The 6 most expensive problems in large .NET teams — eliminated systematically.

- Boilerplate: every feature needs same scaffolding — IA generates in seconds
- Architecture drift: Dev A vs Dev B implement the same pattern differently — IA prevents
- Onboarding: new developer learns from IA output, not tribal knowledge
- Large-scale refactors: change architecture pattern → IA regenerates everything
- Code review overhead: no need to check pattern compliance — IA guarantees it
- Context for AI: structured design gives AI agents accurate

Real customer results (SA & global)

Verified case studies from IA's website — including several South African enterprises you know.

- Teljoy (SA): 2 years done in 6 months — 4× speedup
- DVT (SA, Capgemini): 90% of time on business logic (was 40%)
- PSG (SA): 100% consistency across 30+ microservices with 1 junior
- Aryzac: 12-month rewrite done in 57 hours
- TFG (SA): automated standards enforcement across 40+ applications
- Senwes (SA): 200 hours reduced to 24 hours

How to build tools like IA

The path to tool-builder level. Start with Roslyn analysers — not a full platform.

- Year 1: Roslyn analyser — enforce one architecture rule at compile time
- Year 2: Source generator — IIncrementalGenerator, emit a repository from [Entity]
- Year 3: dotnet new template — team's standard project structure in one command
- Year 4: CLI scaffolder — generate full bounded context from a spec
- Year 5+: Visual tool — domain model → generated code (IA's space)
- Prerequisite: 5+ years senior .NET + deep architecture + compiler API knowledge

12-MONTH STUDY PLAN

12-month study plan: senior dev to solutions architect

Starting from: Senior .NET developer. You already know CQRS, Clean Architecture, Azure basics. Target: AZ-305 certified Solutions Architect with IA module

Q1: C# & SOLID depth

- Apply all 5 SOLID principles to one real class at work — document it as an ADR
- Async deep dive: 'Concurrency in C#' by Stephen Cleary (free online)
- Roslyn: write first analyser — warn when Domain entity has public setter
- AZ-204: 1 hr/day on Microsoft Learn — complete in 8 weeks
- Intent Architect: install, complete Getting Started, generate CRUD module

Q2: Architecture foundations

- AZ-305 study: start now — architecture cert is the career milestone
- Write C4 Context + Container diagram for your Transfer Agency system
- Write 3 ADRs for real decisions made in the last 6 months
- DDD: map one bounded context in your codebase — document in Structurizr
- Multi-tenancy: implement pool model on a side project with EF Core global filter

Q3: Cloud & DevOps

- AZ-305 exam: take and pass — you are now a certified Solutions Architect
- AZ-400: start studying — prove DevOps ownership to employers
- Terraform: provision Azure resources for portfolio project from scratch
- Write source generator: IIncrementalGenerator generating repository from [Entity]
- Full CI/CD pipeline: build from scratch in GitHub Actions — own the YAML

Q4: Architect-level projects

- Build a SaaS product: .NET + Azure + CI/CD + multi-tenancy — real users
- Write Intent Architect custom module for your team's standard pattern
- System design practice: Design Twitter, WhatsApp, Uber — whiteboard sessions
- Structurizr DSL: automate C4 diagrams from code
- Read: 'Designing Data-Intensive Applications' (Kleppmann) — cover to cover

INTERVIEW PREPARATION

Top interview questions: senior dev, SA & EA

Model answers included. The pattern: explain what it is + why it matters + when to use it + a real example from your experience.

Senior Dev

Q: Explain SOLID with an example from your codebase.

A: I applied SRP by splitting our OrderService which handled both business logic and email sending into OrderService + EmailService. The email provider changed later without touching OrderService.

Senior Dev

Q: What is the difference between async/await and parallel programming?

A: Async/await frees threads during I/O waits — one thread serves many requests. Parallel uses multiple threads for CPU work. Use async for DB/HTTP; use Parallel.ForEachAsync for batch CPU work.

Senior Dev

Q: How do you prevent N+1 queries in EF Core?

A: Use .Include() for related data. Use .Select() projection to load only needed columns. Instrument with MiniProfiler or App Insights SQL tracking to detect N+1 in tests.

SA

Q: How do you decide between monolith and microservices?

A: Start with modular monolith. Split when: different teams need independent releases, or one component needs 10× the scale of the rest. Never split prematurely — complexity cost is high.

SA

Q: Explain the CAP theorem and how it applies to your system.

A: CAP: you can only guarantee 2 of Consistency, Availability, Partition Tolerance. For our Transfer Agency (financial data) we chose CP — SQL with ACID transactions. An email notification service could be AP — eventual consistency is fine.

EA

Q: How do you approach a technology strategy for an organisation?

A: Start with business capabilities — what does the organisation need to be able to DO? Then map current systems to capabilities. Identify gaps, duplications, and technical debt. Produce a 3-year roadmap aligned to business objectives.

MASTER CHEAT SHEET

Master reference: terms, certifications & resources

Term / Tool	What it is	Role	Key resource
SOLID	SRP,OCP,LSP,ISP,DIP — 5 OOP design principles	All devs	Clean Code — Uncle Bob
CAP theorem	Consistency vs Availability in distributed systems	SA, senior dev	DDIA — Kleppmann
CQRS	Separate read and write models for different optimisation	Senior dev, SA	Jimmy Bogard blog
DDD	Model complex business domains in code using business language	Senior dev, SA	Blue book — Evans
Clean Arch	Dependencies point inward — Domain knows nothing about infra	Senior dev, SA	Clean Arch — Uncle Bob
Roslyn	C# compiler as a library — parse, analyse, generate code	Tool builder	learn.microsoft.com/dotnet/csharp/roslyn-sdk
Source generator	IIncrementalGenerator — emit .cs files at compile time	Tool builder	Roslyn SDK docs
NetArchTest	Assert Clean Arch rules in unit tests — CI enforcement	Senior dev, SA	NuGet: NetArchTest.Rules
Structurizr	C4 model DSL — living architecture diagrams from code	SA, EA	structurizr.com
TOGAF	Enterprise Architecture framework — TOGAF 10 certification	EA	opengroup.org/togaf
ADR	Architecture Decision Record — documents WHY, not WHAT	SA, EA, senior dev	adr.github.io
AZ-204	Azure Developer Associate	Mid–senior dev	Microsoft Learn
AZ-305	Azure Solutions Architect Expert	SA, Cloud Arch	Microsoft Learn
AZ-400	Azure DevOps Engineer Expert	DevOps	Microsoft Learn
Intent Architect	Architecture-centric AI code gen platform for .NET	All roles	intentarchitect.com
Polly v8	Resilience: retry, circuit breaker, timeout, hedge, bulkhead	Senior dev, SA	pollydocs.org

Reference Additions

.NET Runtime · VSA · AI Engineering · DevOps AI · Books · Security Frameworks · Architecture Thinking

Based on the 9 reference infographics

HOW .NET WORKS

How .NET actually works: Roslyn → IL → JIT → CPU

Most developers never see past 'write C#, run app'. Understanding this pipeline makes you a better debugger, performance engineer, and architect.

The compilation pipeline

C# is never directly executed. It goes through 3 transformations before the CPU sees it.

1 **Roslyn compiles C#**

→ IL (Intermediate Language) in a .dll

2 **CoreCLR loads the .dll**

Assembly Loader resolves deps + Type System loads metadata

3 **JIT compiles IL**

Each method compiled to native code on first call

4 **CPU executes**

Native machine code runs — GC + Thread Pool run alongside

GC Generations

```
Gen0 (short-lived: temp vars) → Gen1 (survived 1 GC) → Gen2 (long-lived: statics, caches) — AVOID Gen2 pressure
```

JIT vs AOT — when each matters

JIT = flexible, warms up at runtime. AOT = fast startup, smaller image — ideal for containers.

	JIT (Just-In-Time)	AOT (NativeAOT)
When	Runtime, on first call	Build time — full native binary
Startup	Slower (JIT overhead)	Fast — no JIT needed
Image size	Larger (IL + runtime)	Smaller — ideal for containers
Flexibility	Dynamic code, reflection OK	Limited reflection support
Best for	Long-running services	Serverless, CLI tools, AKS pods

JIT tiers explained

- Tier 0 (quick JIT): fast compile on first call — not fully optimised
- Tier 1 (re-JIT): hot methods re-compiled with full optimisations after Tier 0 stats collected
- Dynamic PGO: uses runtime behaviour data to optimise further — 10-40% perf gains
- Hot Reload: re-compile changed methods without restart (dotnet watch)
- ReadyToRun: pre-JIT at publish time — faster startup, keeps JIT flexibility

Interview Q: 'What is the difference between JIT and NativeAOT and when would you choose AOT?'

VERTICAL SLICE ARCHITECTURE

VSA vs Clean Architecture: what changes and when to use each

Clean Architecture organises by layer. VSA organises by feature. Both are valid — the right choice depends on complexity and team size.

Clean Architecture

One feature is spread across Domain / Application / Infrastructure / API — 4 projects, many files.

- Organises by LAYER: Domain → Application → Infra → API
- One feature = files in 4 different projects
- Strong separation of concerns — domain protected from infra
- Best when: rich domain logic, complex business rules, DDD needed
- Best when: large teams with clear layer ownership
- Challenge: small change touches files across multiple projects
- Challenge: new developers have to learn the full layer map first
- Interview: 'When would you choose Clean Architecture over VSA?'

Folder structure

```
/Domain/Entities/Order.cs
/Application/Orders/Commands/PlaceOrder/
PlaceOrderCommand.cs
/Application/Orders/Commands/PlaceOrder/
```

Vertical Slice Architecture

One feature = one folder. Everything for that feature lives together — endpoint, handler, mapping, validators.

- Organises by FEATURE: each slice is end-to-end
- One feature = one folder with ALL related files
- Easier to navigate — everything in one place
- Best when: CRUD-heavy apps, medium complexity, smaller teams
- Best when: AI agents need to understand a feature quickly
- No enforced layer boundaries — discipline required
- Scales well with MediatR — each slice is a command/query
- Interview: 'When would you choose VSA over Clean Architecture?'

Folder structure

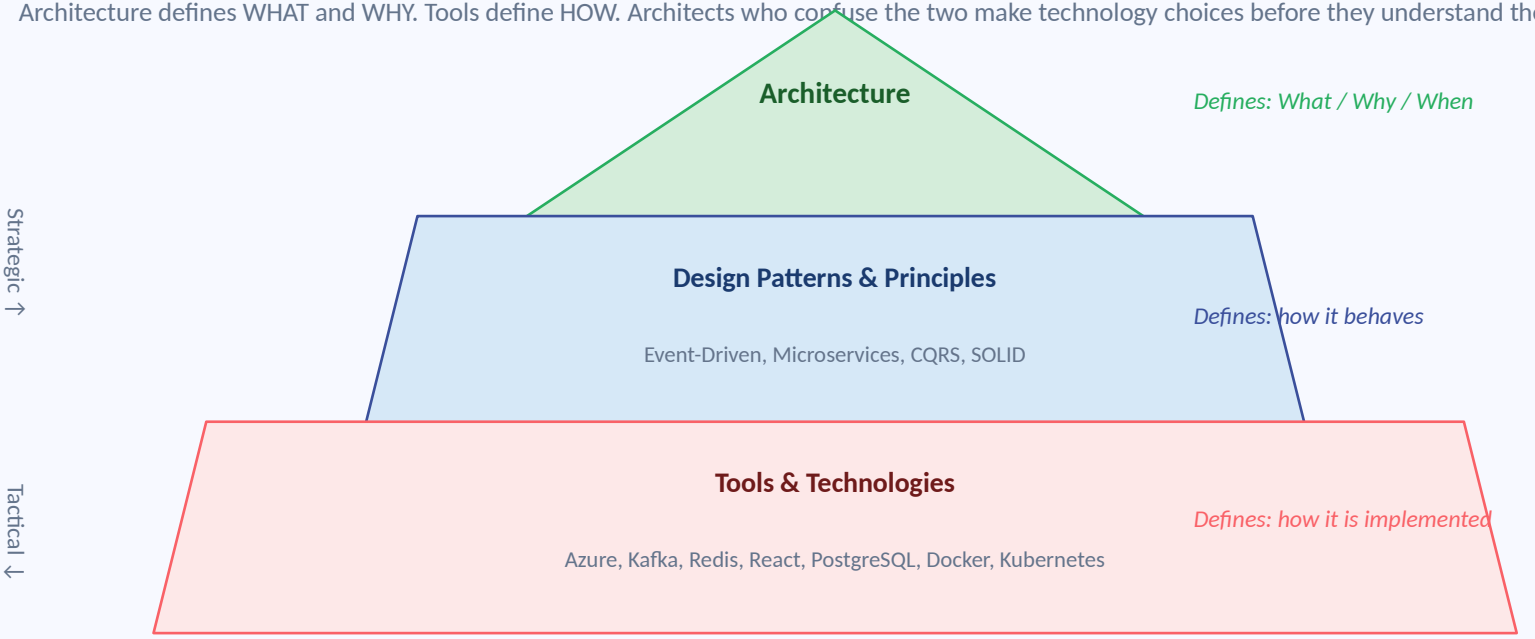
```
/Features/Orders/PlaceOrder/
  PlaceOrderEndpoint.cs   ← all in ONE folder
  PlaceOrderHandler.cs
  PlaceOrderValidator.cs
```

Hybrid (VSA + Clean Architecture): keep slices in the App layer. Add Domain + Infrastructure projects only when complexity justifies it. Best default for growing .NET

ARCHITECTURE vs TOOLS

Architecture vs Tools: the most important mental model

Architecture defines WHAT and WHY. Tools define HOW. Architects who confuse the two make technology choices before they understand the problem.



ARCHITECTURE	TOOLS & TECHNOLOGIES
What / Why / When	How
Long-term decisions (years)	Short-term implementations (months)
Patterns & Principles	Languages, Frameworks, DBs

AI-POWERED DEVOPS

AI-powered DevOps: traditional stack vs AI-augmented stack

AI won't replace DevOps engineers. DevOps engineers using AI will replace those who don't. Know the tools and know the underlying tech they augment.

Kubernetes → **Kagent**

AI Agent for K8s ops — natural language kubectl, anomaly detection

Docker → **Gordon**

AI Assistant for containers — Dockerfile generation, image analysis

GitHub → **Copilot Agent**

AI pair programmer — PR reviews, code generation, test writing

Jenkins / Azure DevOps → **Harness AI**

AI-powered pipeline intelligence — failure prediction, auto-fix

Grafana → **Grafana AI Assistant**

Natural language to dashboards — 'show me p99 latency last 24h'

Prometheus → **HolmesGPT**

AI root cause analysis — correlates metrics, explains incidents

Terraform → **IaC Agent**

AI IaC generation, review & refactoring — from prompt to Bicep/TF

Ansible → **Lightspeed**

AI-powered Ansible automation — playbook generation & optimisation

Key principle: *Understand the underlying tool FIRST (kubectl, terraform, Prometheus) — then use the AI layer on top. The AI tool is only as good as your ability to*

FROM .NET TO AI ENGINEER

.NET to AI Engineer: the skills, the path & the mindset

' NET is my foundation. AI is my force multiplier.' You don't replace your .NET skills — you extend them with AI engineering capabilities.

What you already have (.NET foundation)

Your existing Transfer Agency codebase skills are the foundation. AI engineering builds on top, not instead of.

- C# and .NET — building robust, typed applications
- APIs & Backend Services — REST, middleware, minimal APIs
- Databases — SQL Server, EF Core, indexing, transactions
- Cloud & DevOps — Azure, CI/CD, App Service, Functions
- Messaging — Service Bus, event-driven, background jobs
- Security — auth, authorisation, secrets management
- Strong foundation for enterprise-grade AI systems

AI skills to add next

These are the concrete skills that bridge .NET developer to AI-enabled engineer. Learn them one project at a time.

- Prompting & context engineering — structured prompts, few-shot examples
- RAG (Retrieval-Augmented Generation) — retrieve relevant docs, augment LLM
- Embeddings & semantic search — vectors, cosine similarity, meaning-based search
- Vector databases — Pinecone, Chroma, Azure AI Search — store embeddings
- Chunking strategies — split documents optimally for retrieval quality
- AI in engineering workflows — connect AI outputs to real .NET SDLC

The progression (where you are → goal)

Each stage is real work, not just theory. You move forward by building things, not just reading about them.

- .NET Engineer: strong software engineering foundation in .NET ecosystem
- AI-assisted Engineer: using Copilot, ChatGPT to boost productivity
- AI-enabled Engineer: building workflows using RAG, embeddings, tools ← YOU ARE HERE
- AI Engineer (goal): design reliable, secure, scalable AI-powered systems
- Key: not just using AI to write code — building systems where AI adds real value
- One concept at a time. One project at a time. One step closer to the future.

What to apply and deepen next

These are the focus areas that move you from AI-enabled to full AI Engineer capability.

- 1. Advanced RAG: multi-step, hybrid search, query rewriting, re-ranking
- 2. AI output evaluation: quality, correctness, relevance metrics
- 3. AI Agents & tool calling: plan → use tools → execute multi-step workflows
- 4. MCP & enterprise integrations: AI + Atlassian, Azure DevOps, enterprise APIs
- 5. Guardrails & Human-in-the-Loop: safety policies, risk-based approvals
- 6. Observability & cost control: token usage, latency, failure rates
- 7. AI Architecture Patterns: scalable, secure, maintainable AI in production

ESSENTIAL READING

12 books that will make you an expert engineer

Recommended by the HN / Reddit / ACM engineering community. Read in order — each builds on the previous. DDIA is the single most important book for your

Priority #1 — **Designing Data-Intensive Applications**
Kleppmann

Modern databases, distributed systems, CAP theorem, consistency — the architect's bible

Month 1 — **Clean Code**
Robert C. Martin

Readable, maintainable code — meaningful names, focused functions, refactor continuously

Month 2 — **The Pragmatic Programmer**
Hunt & Thomas

Think in systems, automate repetitive work, continuously learn — mindset for senior devs

Month 2 — **The Mythical Man-Month**
Fred Brooks

Why large software projects struggle — Brooks's Law, communication paths, conceptual integrity

Month 3 — **Clean Architecture**
Robert C. Martin

Dependency rule, layers, boundaries — the theory behind Clean Architecture in .NET

Month 3 — **Code Complete 2**
Steve McConnell

Practical techniques for high-quality software — write maintainable code, prevent defects

Month 4 — **Refactoring**
Martin Fowler

Improve code without changing behaviour — remove smells, simplify complex logic, small steps

Month 4 — **Domain-Driven Design**
Eric Evans

The 'blue book' — entities, value objects, bounded contexts, ubiquitous language

Month 5 — **Peopleware**
DeMarco & Lister

How teams, culture, and environment affect productivity — essential for tech lead path

Month 6 — **Design Patterns (GoF)**
Gang of Four

The 23 classic patterns — factory, decorator, strategy, observer, command — with structure

Advanced — **The Art of Computer Programming**
Donald Knuth

Deep foundations of algorithms and mathematical analysis — long-term investment

Advanced — **Enterprise Integration Patterns**
Hohpe & Woolf

Integration patterns: message channels, routers, transformers — essential for EA path

SECURITY FRAMEWORKS

Top security architecture frameworks: when to use each

Not every framework suits every organisation. For most .NET Azure developers: Zero Trust is your daily practice. NIST CSF for posture. ISO 27001 for compliance.

Framework	Primary Focus	Best For	Relevance to You
Zero Trust Architecture	Identity-centric security — never trust, always verify	Cloud-native and hybrid environments — Azure AD + Managed Identity IS Zero Trust	High — you are already applying it: JWT, Managed Identity, private endpoints
NIST CSF 2.0	Cybersecurity risk management	Any org improving security maturity — broad industry adoption	Medium — understand the 6 functions: Govern, Identify, Protect, Detect, Respond, Recover
ISO 27001	Information security management — ISMS	Compliance-focused orgs needing global credibility	High for SA — Transfer Agency clients may require ISO 27001 evidence from vendors
TOGAF	Enterprise architecture governance — not just security	Enterprise transformation programmes — org-wide standards	High for EA path — TOGAF 10 certification is your EA milestone
SABSA	Business-driven security architecture — risk context	Large enterprises, regulated industries (financial, healthcare)	Medium — Transfer Agency sector; understand risk attributes and traceability
COBIT	Governance and compliance management	Boards and audit-driven enterprises	Low-medium — relevant when you reach EA level and need governance language
NIST 800-53	Security controls catalogue	US federal / high-security environments	Low for SA context — but NIST CSF 2.0 (above) uses similar language
Gartner CARTA	Adaptive trust and continuous risk assessment	AI-driven, dynamic environments where static rules fail	Medium — the future direction; relevant for AI-enabled systems you'll build

PATTERNS IN PRODUCTION

Production patterns: Result, Specification, Chain of Responsibility & Adapter

These four patterns appear constantly in real .NET systems but are underrepresented in interview prep. Know them with code.

Result pattern — errors as values

Return success or failure without throwing exceptions for flow control.

```
public Result<Order> PlaceOrder(PlaceOrderCommand
cmd) {
  if (!cmd.Items.Any())
    return Result.Failure<Order>("Order must have
items");
  var order = Order.Create(cmd);
  return Result.Success(order);
}
// Caller: if(result.IsSuccess) ... else
```

Specification pattern — encapsulate queries

Encapsulate a query predicate as a named, reusable, composable object.

```
public class ActiveOrdersForCustomer :
Specification<Order> {
  public ActiveOrdersForCustomer(int customerId)
    => Query.Where(o => o.CustomerId == customerId
                   && o.Status ==
OrderStatus.Active)
            .OrderByDescending(o => o.CreatedAt);
}
// Usage: await _repo.ListAsync(new
```

Chain of Responsibility — MediatR pipelines

Pass requests through a chain of handlers. Each handler decides to

```
// MediatR pipeline = Chain of Responsibility
// Request flows: LoggingBehaviour →
ValidationBehaviour → Handler
// Each behaviour calls 'await next()' to pass to
the next link
// This IS the Chain of Responsibility pattern —
named and applied
// Add new cross-cutting concern = add new pipeline
behaviour
```

Adapter pattern — translate between interfaces

Make incompatible interfaces work together. Wrap a third-party service

```
// Your interface (defined in Application layer)
public interface IEmailService { Task
SendAsync(Email email); }

// Adapter wraps SendGrid SDK (Infrastructure
layer)
public class SendGridAdapter : IEmailService {
  public async Task SendAsync(Email email) {
    var msg = new SendGridMessage(); // translate
```

Resources: 'Design Patterns That Deliver' — thecodeman.net | Free chapter: Builder Pattern in Modern C# | All patterns shown in real production .NET code

SA INTERVIEW FRAMEWORK

Solutions Architect interview framework: the 14-step process

When given a system design problem in an interview, follow this exact sequence. It demonstrates architect thinking, not just technical knowledge.

1 Clarify Requirements

Functional (what it does) + NFRs (how well it does it) — never skip this

2 Estimate Scale

Users, requests/sec, data volume, read/write ratio — drives all architecture decisions

3 Design APIs

REST endpoints, request/response shapes, auth model, versioning strategy

4 Design Data Model

SQL vs NoSQL, schema, indexes, partitioning, tenancy model

5 High-Level Architecture

CDN → WAF → Load Balancer → API Gateway → Services → Message Bus → Data

6 Design Architecture

Monolith vs microservices, sync vs async, bounded contexts, service boundaries

7 Plan Reliability

Circuit breaker, retry, timeout, bulkhead, DLQ, saga pattern

8 Plan Security

Zero Trust, JWT, RBAC, Key Vault, encryption in transit + at rest

9 Plan Observability

Structured logging, distributed tracing, metrics, alerting, dashboards

10 Discuss Trade-offs

Cost vs complexity vs performance — explicitly state what you're optimising for

11 Estimate Cost

Rough Azure cost — compute, storage, messaging, bandwidth — shows business awareness

12 Summarise & Validate

Repeat back the decisions, confirm NFRs are met, invite questions

Key architect vocabulary: High Availability · Fault Tolerance · Resilience · Scalability · Elasticity · Observability · Event-Driven · Loose Coupling · Domain-Driven Design · Cloud Native · Zero

COMPLETE INDEX

Complete handbook index — all 49 slides

Use this as your navigation guide. Each section is self-contained — jump to what you need